

TABLE IV
Gray/Reflected Code

Decimal	Reflected decimal	Binary $b_3 b_2 b_1 b_0$	Reflected binary/gray code $g_3 g_2 g_1 g_0$
00	00	0000	0000
01	01	0001	0001
02	02	0010	0011
03	03	0011	0010
04	04	0100	0110
05	05	0101	0111
06	06	0110	0101
07	07	0111	0100
08	08	1000	1100
09	09	1001	1101
10	19	1010	1111
11	18	1011	1110
12	17	1100	1010
13	16	1101	1011
14	15	1110	1001
15	14	1111	1000
16	13		
17	12		
18	11		
19	10		

TABLE V
Illustration of Property of Gray Code

Present number	Action	Next number
0011 (=3) Starting position	Change in the first position from right	0010 (=2)
0010(=2)	Change in the second position from right	0000(=0)
0000(=0)	Change in the third position from right	0100(=4) Final position

Comment: So for a change from regulating position 3 to 4, the fan will go a round of 3 to 2 to 0 to 4. This will cause huge oscillation in the circuit, which is not a good design.

us take the example of a fan regulator which is digitally controlled. You want to switch from position 3 to 4. If you follow the normal binary system, you need to change three bit positions to switch from 3 to 4. But how? Change three positions one after another as shown in Table V.

In gray code, there is change in only one position, so there will be no oscillation as such. The code is called reflected because it can be generated in the following manner: Take the gray code 0, 1. Write it forwards, then backwards: 0, 1, 1, 0. Then prepend 0's

to the first half and 1's to the second half: 00, 01, 11, 10. Continuing, write 00, 01, 11, 10, 10, 11, 01, 00 to obtain 000, 001, 011, 010, 110, 111, 101, 100. Each iteration therefore doubles the number of codes.

Algorithm for binary-to-gray conversion

This is very simple:

1. The most significant bit of the binary number is the most significant bit of the gray code

2. Add (using modulo 2, i.e., ignoring carry) the next significant bit of the binary number to the next significant bit of the binary number to obtain the next gray code bit.

3. Repeat step 2 until all bits of the binary bits have been added modulo 2. The resultant number is

equivalent binary is 0011.

Excess-3 gray code

In many applications, it is desirable to use a BCD as well as unit distance. Excess-3 gray code is such a code. The values for 0 and 9 differ in only one bit, and so do all values for successive numbers. Outputs from linear devices or angular encoders may be coded in excess-3 gray code to obtain multi-digit BCD numbers.

The code obtained from excess-3 code by applying conversion rule is shown below:

Decimal	Excess-3 gray code
0	0010
1	0110
2	0111
3	0101
4	0100
5	1100
6	1101
7	1111
8	1110
9	1010

Error correction and detection codes

When a binary message made of strings of 0's and 1's is transmitted from the source to the destination, the message is corrupted by the noise during transmission. The corrupted message becomes erroneous by conversion from transmitted 0 to received 1 or from transmitted 1 to received 0.

The error-correction code (ECC) and the error-detection code (EDC) are used to rectify the errors. In EDCs and ECCs, redundant check bits are appended with original message bits to design the codes. EDCs detect presence of errors. ECCs detect and correct the errors.

Typically, transmission errors are of two types: random and burst. An error is called random if bits in the error are randomly distributed over the code. Burst error occurs when bits in the error are clustered together over the code. For transmitted byte 01010101, the examples of random error and burst error may be as below

the gray code equivalent of the binary number.
Assume binary number $b_3 b_2 b_1 b_0 = 0011$. Conversion into gray is done as follows: $g_3 = b_3 = 0$; $g_2 = b_3 + b_2 = 0 + 0 = 0$; $g_1 = b_2 + b_1 = 1 + 0 = 1$; and $g_0 = b_1 + b_0 = 1 + 1 = 0$ (ignore 'carry')

Gray-to-binary conversion

Assume gray code $g_3 g_2 g_1 g_0 = 0010$. It can be converted into binary as follows: $b_3 = g_3 / 2 = \text{remainder } 0$; $b_2 = [g_3 + g_2] / 2 = \text{remainder } 0$; $b_1 = [g_2 + g_1] / 2 = \text{remainder } 1$; $b_0 = [g_1 + g_0] / 2 = \text{remainder } 1$. Thus